

Design and Implementation of a Database Management System for Airline Reservation and Flight Operations Management

Submitted by

Hrishika Birru | 25WU0101047

Sukriti Gupta | 25WU0102323

Mithila | 25WU0102327

Department of Computer Science and Engineering

Course: Database Management Systems

Faculty: Dr. Kiran Mayee

Academic Year: 2026–2027

Table of Contents

Abstract.....	3
I. Introduction	4
II. Problem Statement	5
III. Objectives and Scope	6
A. Objectives	6
B. Scope.....	6
IV. System Requirements.....	7
A. Software Requirements.....	7
B. Hardware Requirements.....	7
V. System Design	8
A. Entity-Relationship Diagram	8
B. Relational Schema	8
C. Normalization	9
D. Data Dictionary.....	9
VI. Implementation.....	10
A. Technology Stack	10
B. SQL Commands.....	10
C. Database Connectivity	11
D. Passenger Backend and CRUD Operations	11
E. Validation and Error Handling.....	12
F. System Architecture and Data Flow	12
G. User Interface.....	12
VII. Testing.....	14
VIII. Conclusion and Future Work.....	15
Contribution of Team Members.....	16
Appendix A: Source Code Repository	17
References.....	18

Abstract

The Airline Reservation and Flight Operations Management System is a database-driven application that manages passengers, airports, aircraft, flights, seats, reservations, and payments. The system uses MySQL as the backend database, Python for backend processing and database connectivity, and Streamlit for the user interface. The database is designed using relational concepts, primary keys, foreign keys, unique constraints, and normalization to maintain data consistency and reduce redundancy. Python backend modules provide CRUD (Create, Read, Update, Delete) operations along with validation and error handling. The project demonstrates the practical use of DBMS concepts such as database design, SQL queries, normalization, integrity constraints, connectivity, and application-level database operations.

Index Terms: Airline reservation, CRUD operations, MySQL, normalization, Python, relational database, Streamlit.

I. Introduction

Airline reservation systems handle large amounts of interconnected information, including passenger details, flight schedules, airports, aircraft, seat availability, reservations, and payments. Managing this information manually or in separate data sources leads to data redundancy, inconsistency, and difficulty in retrieving records.

To address these issues, an Airline Reservation and Flight Operations Management System was developed using a relational database. Airline information is stored in structured tables, and relationships between entities are established using primary and foreign keys. A Python backend is integrated with the MySQL database, and a Streamlit-based interface lets users perform database operations without writing SQL commands directly.

II. Problem Statement

The main problem addressed by this project is the efficient management of airline reservation and flight operation data. Without a properly designed database, maintaining passenger records, flight information, reservations, and payment details can result in duplication, inconsistent records, and difficulty in retrieving and updating information.

The project therefore aims to design and implement a centralized relational database system that provides structured storage, maintains relationships between entities, supports CRUD operations, and allows users to interact with the data through a simple application interface.

III. Objectives and Scope

A. Objectives

- To design a relational database for airline reservation and flight operations.
- To identify entities and establish relationships between them.
- To implement the database using MySQL.
- To apply primary keys, foreign keys, unique constraints, and other integrity constraints.
- To implement CRUD operations using Python and establish Python–MySQL connectivity.
- To provide a simple Streamlit-based user interface.
- To execute SQL queries for retrieving and manipulating data.
- To maintain data consistency and reduce redundancy through normalization.
- To test the system using valid and invalid inputs.

B. Scope

The system manages passenger information, airport details, aircraft information, flight schedules, seat information, reservations, and payment records. It supports insertion, retrieval, updating, and deletion of records, with application-level interaction through a Streamlit interface.

The current version focuses on database management and application integration. Future versions may include user authentication, online payment integration, advanced seat selection, real-time flight tracking, and cloud deployment.

IV. System Requirements

A. Software Requirements

Requirement	Technology
Operating System	Windows
Database	MySQL Server 8.0
Programming Language	Python
Database Connector	mysql-connector-python
User Interface	Streamlit
IDE	VS Code
Version Control	Git and GitHub

B. Hardware Requirements

Requirement	Minimum
Processor	Intel i3 or equivalent
RAM	4 GB or above
Storage	10 GB available
Internet	Required for GitHub and package installation

V. System Design

A. Entity-Relationship Diagram

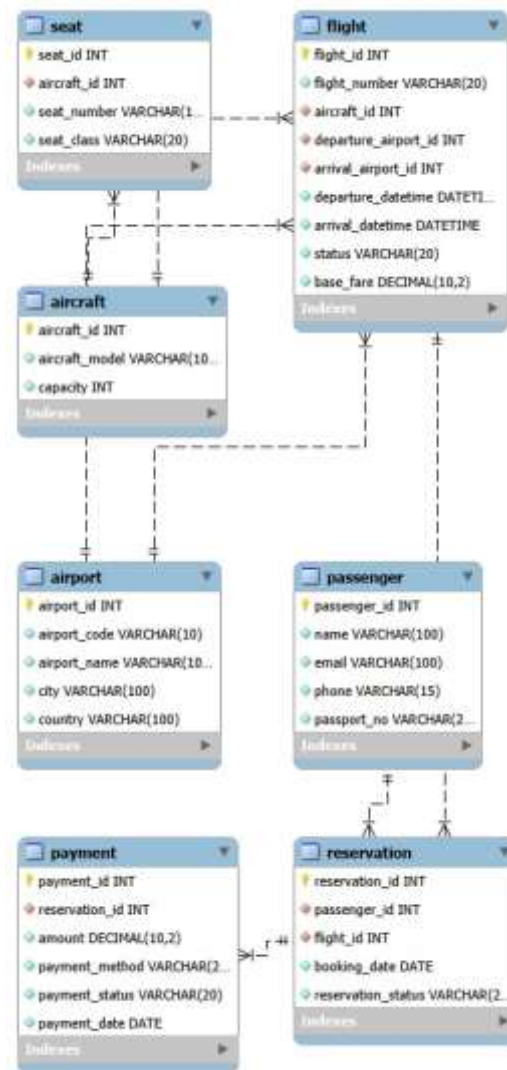


Fig. 1. Entity-Relationship diagram of the Airline Reservation System.

The ER diagram represents the major entities of the system: Passenger, Airport, Aircraft, Flight, Seat, Reservation, and Payment. Primary keys uniquely identify records, while foreign keys establish relationships between entities. The key relationships are:

- **Passenger – Reservation:** A passenger can make one or more reservations; each reservation belongs to one passenger.
- **Flight – Reservation:** A flight can have many reservations; each reservation is for one flight.
- **Reservation – Payment:** A reservation can be associated with payment information.
- **Aircraft – Flight:** An aircraft can be assigned to multiple flights over time.

B. Relational Schema

The ER model was converted into the following relations. Primary keys (PK) and foreign keys (FK) are noted where applicable.

TABLE I. RELATIONAL SCHEMA

Relation	Attributes	Keys
Passenger	passenger_id, name, email, phone, ...	PK: passenger_id
Airport	airport_id, airport_name, city, country, ...	PK: airport_id
Aircraft	aircraft_id, model, capacity, ...	PK: aircraft_id
Flight	flight_id, aircraft_id, source_airport, destination_airport, departure_time, arrival_time, ...	PK: flight_id; FK: aircraft_id, source and destination airport
Seat	seat_id, aircraft_id, seat_number, class, ...	PK: seat_id; FK: aircraft_id
Reservation	reservation_id, passenger_id, flight_id, seat_id, ...	PK: reservation_id; FK: passenger_id, flight_id, seat_id
Payment	payment_id, reservation_id, amount, payment_status, ...	PK: payment_id; FK: reservation_id

Note: attribute lists should be matched to the final SQL script before submission.

C. Normalization

The database was designed using normalization principles to reduce redundancy and improve data consistency.

- **First Normal Form (1NF):** All attributes hold atomic values and repeating groups are avoided.
- **Second Normal Form (2NF):** Non-key attributes depend on the complete primary key, so partial dependencies are avoided.
- **Third Normal Form (3NF):** Non-key attributes depend only on the primary key, which removes transitive dependencies.

Normalization helps prevent insertion, deletion, and update anomalies while keeping the relational structure consistent.

D. Data Dictionary

Table II shows the data dictionary for the Passenger table. Equivalent tables should be given for Airport, Aircraft, Flight, Seat, Reservation, and Payment using the final SQL script.

TABLE II. PASSENGER TABLE

Column	Data Type	Constraint	Description
passenger_id	INT	PRIMARY KEY	Unique passenger ID
name	VARCHAR(100)	NOT NULL	Passenger name
email	VARCHAR(100)	UNIQUE	Passenger email
phone	VARCHAR(15)	—	Contact number

VI. Implementation

A. Technology Stack

The system uses MySQL for database management, Python for backend development, mysql-connector-python for database connectivity, and Streamlit for the user interface.

B. SQL Commands

1) DDL Commands: Data Definition Language commands were used to create the database structure.

```
CREATE DATABASE airline_reservation_db;  
  
CREATE TABLE Passenger (  
    passenger_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    phone VARCHAR(15)  
);
```



Fig. 2. MySQL output showing the created tables.

2) DML Commands: Data Manipulation Language commands were used to insert, retrieve, update, and delete records.

```
INSERT INTO Passenger VALUES (...);  
SELECT * FROM Passenger;  
UPDATE Passenger SET phone = '9871262373' WHERE passenger_id = 1;  
DELETE FROM Passenger WHERE passenger_id = 6;
```

Result Grid					
Filter Rows:					
	passenger_id	name	email	phone	passport
▶	1	Rahul Sharma	rahul.sharma@gmail.com	9876543210	P123456
	2	Priya Reddy	priya.reddy@gmail.com	9876543211	P123456
	3	Arjun Mehta	arjun.mehta@gmail.com	9876543212	P123456
	4	Ananya Singh	ananya.singh@gmail.com	9876543213	P123457
	5	Vikram Rao	vikram.rao@gmail.com	9876543214	P123457
•	NULL	NULL	NULL	NULL	NULL

Fig. 3. Output of the DML commands.

3) Sample Queries: The following queries were executed to retrieve related data.

```
-- Q1: Retrieve all passengers
SELECT * FROM Passenger;

-- Q2: Passenger and reservation details
SELECT p.name, r.reservation_id
FROM Passenger p
JOIN Reservation r ON p.passenger_id = r.passenger_id;

-- Q3: Flight details with aircraft
SELECT ...
FROM Flight f
JOIN Aircraft a ON f.aircraft_id = a.aircraft_id;
```

	name	reservation_id
▶	Rahul Sharma	1
	Priya Reddy	2
	Arjun Mehta	3
	Ananya Singh	4
	Vikram Rao	5

Fig. 4. Query outputs for Passenger and reservation details

C. Database Connectivity

A Python connection module (`database.py`) was created to establish communication between the application and the MySQL database using the `mysql.connector` library.

```
import mysql.connector

def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="*****",
        database="airline_reservation_db"
    )
```

The `get_connection()` function creates and returns a connection to the database. Other backend modules call this function whenever they need to run a database operation.

D. Passenger Backend and CRUD Operations

A separate Python module handles passenger-related database operations. Its functions execute SQL statements and commit the required changes to the database.

TABLE III. CRUD FUNCTIONS OF THE PASSENGER MODULE

Operation	Function	Purpose
Create	<code>add_passenger()</code>	Adds a new passenger
Read	<code>get_passengers()</code>	Retrieves passenger records
Update	<code>update_passenger()</code>	Updates passenger information
Delete	<code>delete_passenger()</code>	Deletes a passenger record

E. Validation and Error Handling

Validation and exception handling were added to manage invalid database operations. For example, deleting a passenger who is referenced by a reservation violates a foreign key constraint (MySQL Error 1451). The backend catches such errors instead of letting the application terminate unexpectedly. This behavior also demonstrates that referential integrity is correctly enforced by the database.

F. System Architecture and Data Flow

```
Streamlit UI
|
v
Python Backend Modules
|
v
```

```

database.py
|
v
MySQL Connector
|
v
MySQL Database

```

When a user performs an operation in the Streamlit interface, the request is passed to the Python backend. The backend opens a database connection and executes the corresponding SQL operation, and the result is returned to the interface.

G. User Interface

The Streamlit interface provides the following pages:

- **Home page:** Entry point for the system (Fig. 5).
- **Passenger management:** Add, view, update, and delete passenger records (Fig. 6).
- **Flight management:** Manage flight records (Fig. 7).
- **Reservation and payment:** Manage bookings and payment details (Fig. 8).

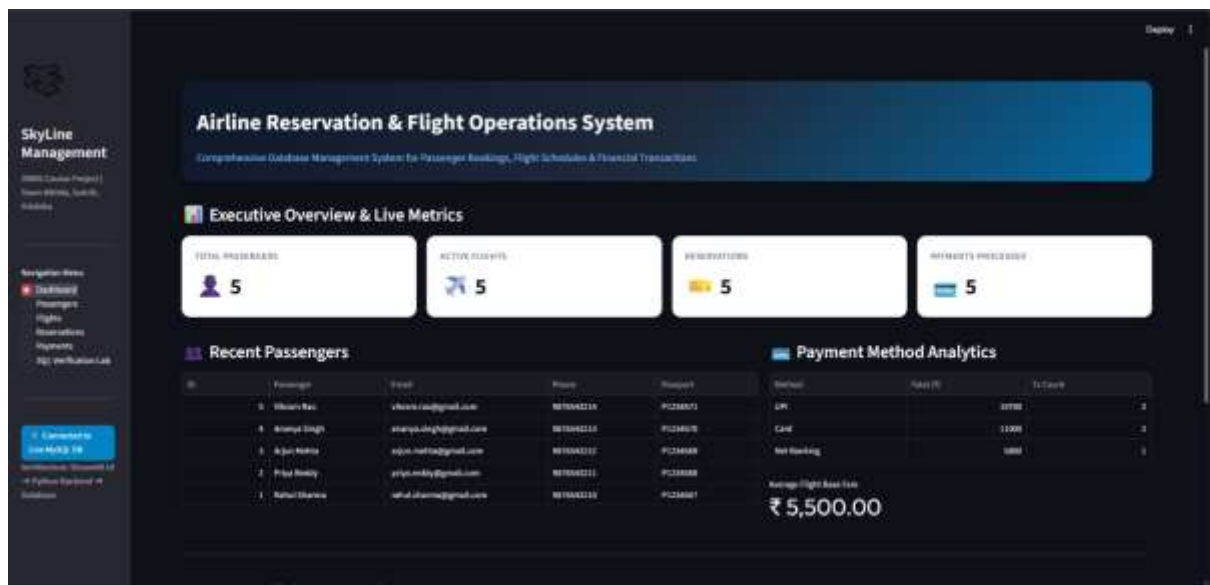


Fig. 5. Home page of the Streamlit application.

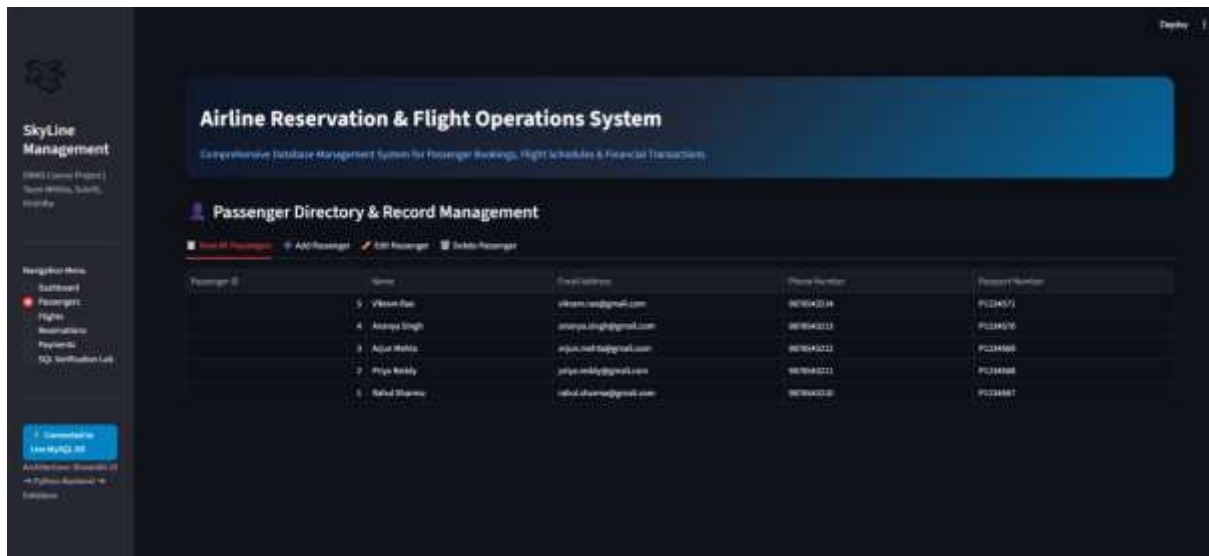


Fig. 6. Passenger management page.

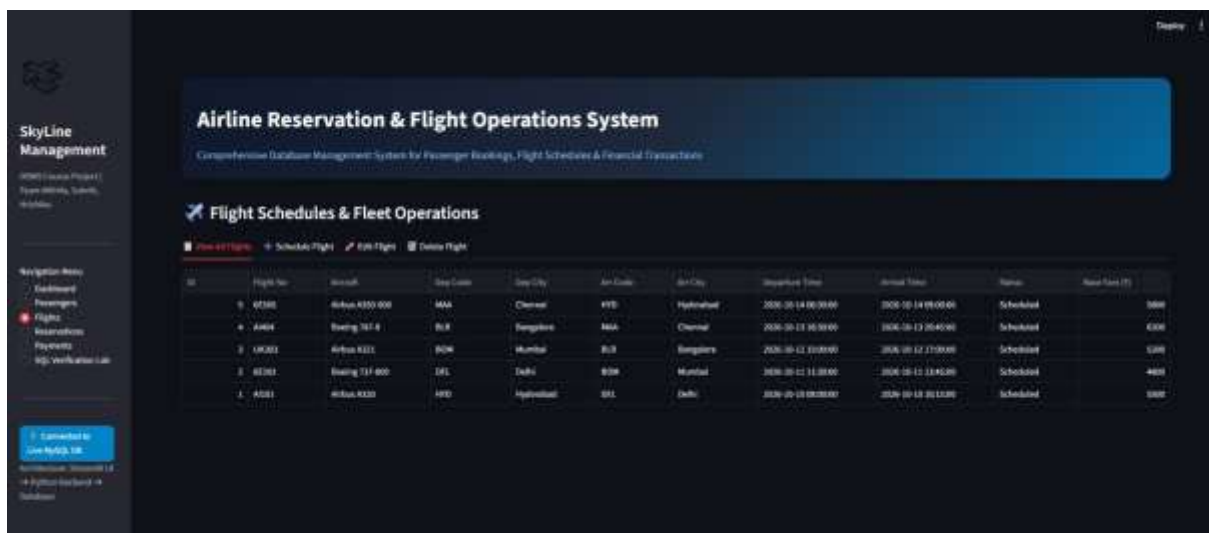


Fig. 7. Flight management page.

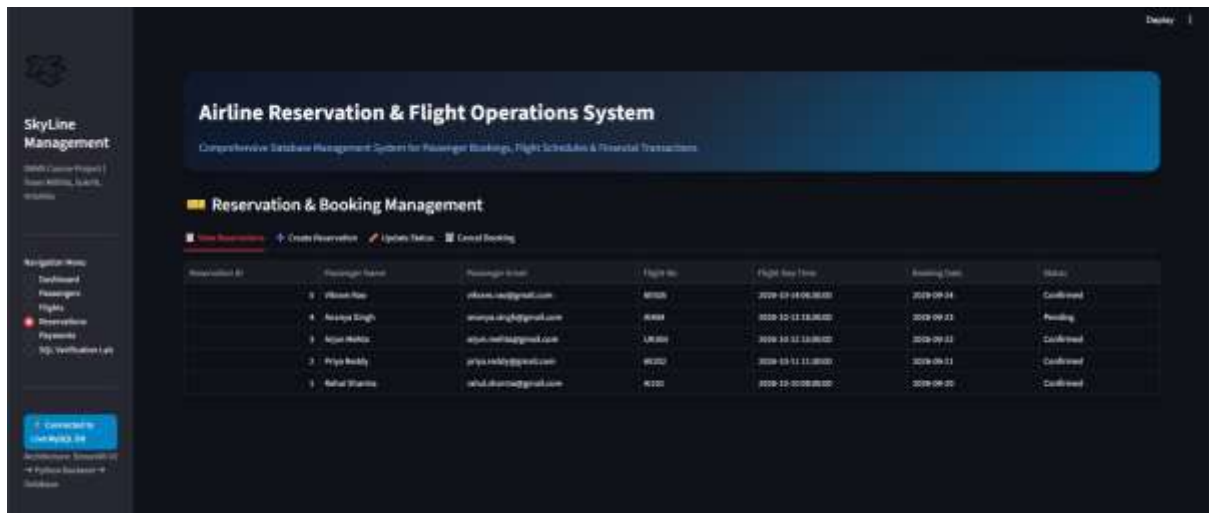


Fig. 8. Reservation page.

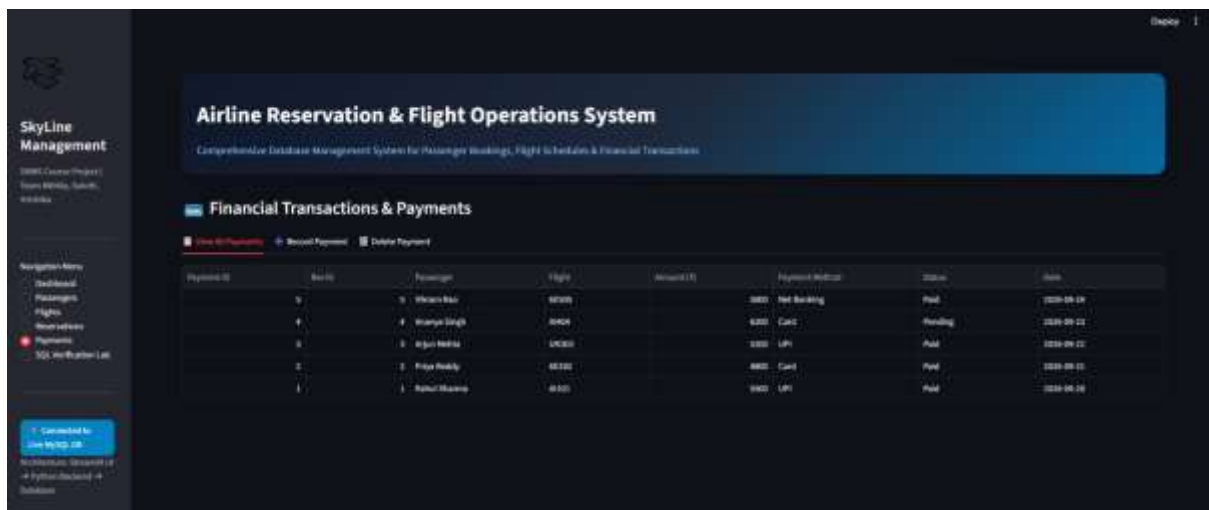


Fig. 9. Pyment page.

VII. Testing

The system was tested using valid and invalid inputs, including constraint-related cases.

TABLE IV. TEST CASES AND RESULTS

Test Case	Input / Action	Expected Result	Result
Add passenger	Valid passenger details	Passenger inserted	Pass
Retrieve passengers	Select passenger records	Records displayed	Pass
Update passenger	Change phone or email	Record updated	Pass
Delete passenger	Delete valid passenger	Record removed	Pass
Duplicate email	Existing email	Constraint error	Pass
Invalid passenger ID	Non-existing ID	Error / no record	Pass
Delete referenced passenger	Passenger with reservation	Foreign key error	Pass

VIII. Conclusion and Future Work

The Airline Reservation and Flight Operations Management System demonstrates the practical implementation of database management concepts using MySQL and Python. It provides structured storage and management of passenger, flight, airport, aircraft, reservation, seat, and payment information. The project applies relational design, normalization, SQL commands, constraints, CRUD operations, database connectivity, and application-level interaction through Streamlit. The Python backend acts as an intermediate layer between the interface and the database, allowing records to be managed efficiently.

Future enhancements include:

- User authentication and role-based access
- Real-time flight status
- Advanced seat selection
- Online payment gateway
- Email/SMS reservation confirmation
- Flight search and filtering
- Cloud deployment
- Admin dashboard, reports, and analytics

Contribution of Team Members

- **Hrishika Birru:** Developed the Python backend modules, implemented MySQL connectivity using mysql-connector-python, developed CRUD operations, handled database exceptions and validation, integrated the backend with the application interface, and contributed to testing and documentation.
- **Sukriti Gupta:** Designed the database structure, prepared SQL scripts, developed the ER diagram, created sample data, and contributed to database implementation and query development.
- **Mithila:** Developed the Streamlit user interface, integrated it with the backend, and contributed to application testing and UI refinement.

Appendix A: Source Code Repository

The complete source code, database scripts, application modules, and related project files are available at: <https://github.com/sukriti-8/DBMS-Course-Project.git>

References

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [2] Oracle Corporation, “MySQL 8.0 Reference Manual.” [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [3] Oracle Corporation, “MySQL Connector/Python Developer Guide.” [Online]. Available: <https://dev.mysql.com/doc/connector-python/en/>
- [4] Python Software Foundation, “Python Documentation.” [Online]. Available: <https://docs.python.org/3/>
- [5] Streamlit Inc., “Streamlit Documentation.” [Online]. Available: <https://docs.streamlit.io/>
- [6] S. Gupta et al., “DBMS Course Project,” GitHub repository. [Online]. Available: <https://github.com/sukriti-8/DBMS-Course-Project.git>